LAPORAN UJIAN TENGAH SEMESTER
KOMPUTASI PARALEL DAN TERDISTRIBUSI



Oleh:

Nama : Abang Malik Syaidar
NIM : 231220010
Kelas: 31

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK DAN ILMU KOMPUTER
UNIVERSITAS MUHAMMADIYAH PONTIANAK
2026

1. Tujuan Praktikum

Praktikum UTS ini bertujuan untuk: (1) menjelaskan perbedaan concurrency, paralelisme, dan distribusi pada aplikasi nyata; (2) mengimplementasikan REST API asinkron dengan FastAPI dan SQLite melalui `aiosqlite`; (3) memproses transaksi di latar belakang dengan `BackgroundTasks`; (4) mencegah race condition pada pengurangan stok dengan `asyncio.Lock`; dan (5) membandingkan performa 1 worker dengan 4 worker Uvicorn menggunakan Locust.

Deskripsi aplikasi

Aplikasi yang dibangun adalah **API Toko Buku**. Resource utamanya adalah `books` (id, judul, penulis, kategori, harga, stok) dan resource transaksinya adalah `transactions` (id, item_id, qty, total_harga, status, catatan, created_at). Fitur utama: (a) CRUD buku dengan paginasi `skip` dan `limit` serta hard delete; (b) pembuatan transaksi yang langsung dijawab berstatus `pending`, lalu diproses di latar belakang dengan alur `pending → processing → completed/failed`; (c) pemotongan stok yang aman dari race condition.

Batasan sistem: SQLite hanya mengizinkan satu penulis pada satu waktu; `asyncio.Lock` hanya berlaku di satu proses (pada multi-worker dilengkapi guard `UPDATE ... WHERE stok >= qty`); pembayaran hanya disimulasikan dengan `asyncio.sleep(3)`; belum ada autentikasi; dan pengujian dilakukan pada satu laptop sehingga Locust dan server berbagi CPU.

2. Dasar Teori

a. Bagian yang bersifat concurrent (I/O-bound)

Seluruh akses basis data melalui `aiosqlite` (misalnya `get_book`, `insert_transaction`, `update_trx_status`) dan `asyncio.sleep(3)` pada `proses_transaksi_background` adalah pekerjaan I/O-bound. Saat satu coroutine menunggu (`await`), event loop menjalankan request lain di thread yang sama. Banyak request dapat berjalan bergantian (concurrent) tanpa membuat thread baru.

b. Bagian yang berpotensi membutuhkan paralelisme (CPU-bound)

Pada versi sekarang hampir tidak ada, karena validasi Pydantic dan serialisasi JSON tergolong ringan. Paralelisme berbasis proses (multiprocessing) baru diperlukan bila ditambahkan fitur berat seperti pembuatan laporan penjualan besar atau perhitungan rekomendasi buku.

c. GIL dan multi-worker Uvicorn

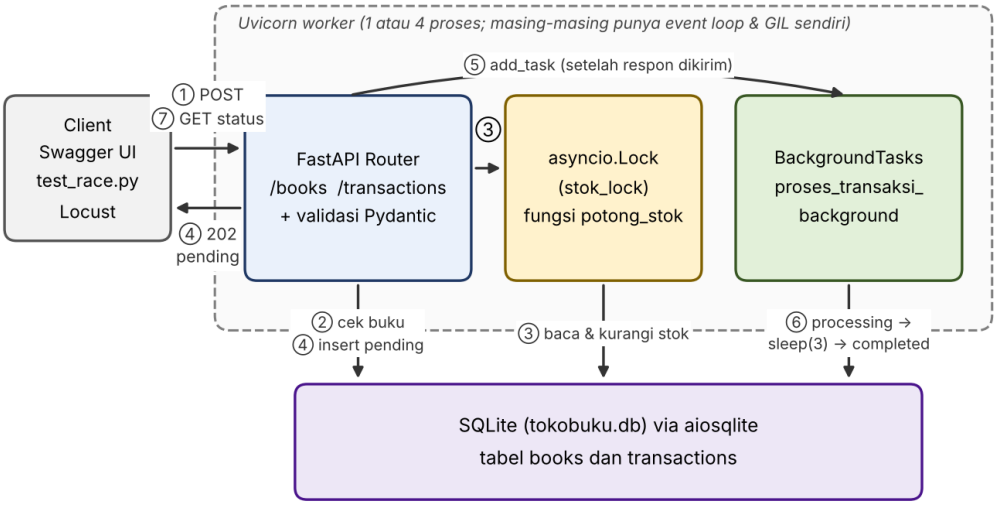
Python GIL (Global Interpreter Lock) hanya mengizinkan satu thread menjalankan bytecode Python pada satu waktu dalam satu proses, sehingga threading tidak mempercepat pekerjaan CPU-bound (hanya membantu pekerjaan I/O-bound). Opsi `--workers N` pada Uvicorn membuat N proses terpisah; masing-masing memiliki interpreter, GIL, dan event loop sendiri, sehingga request dapat dilayani paralel di beberapa core CPU. Konsekuensinya, memori tidak dibagi antar worker, sehingga `asyncio.Lock` hanya melindungi satu proses.

3. Langkah Kerja

3.1 Langkah pengujian

Race condition. Server dijalankan dengan `USE_LOCK=0` (tanpa lock) lalu `USE_LOCK=1` (dengan `stok_lock`); pada kedua mode, `test_race.py` mengirim dua request serentak untuk buku berstok 1, dan fungsi pemotongan stok memiliki jeda `asyncio.sleep(0.01)` setelah membaca stok untuk membuka jendela race condition. **Benchmark.** Server dijalankan dengan `--workers 1` dan `--workers 4`. Locust (headless) menjalankan Read `GET /books` (100 request, konkurensi 20) dan Write `POST /transactions` (50 request, konkurensi 10), masing-masing 3 kali, dan dilaporkan nilai tengahnya (median).

3.2 Arsitektur dan alur data



Gambar 1. Arsitektur aplikasi dan alur satu siklus transaksi (① sampai ⑦)

Alur satu siklus transaksi lengkap (sampai status `completed`):

1. Klien mengirim `POST /transactions`; Pydantic memvalidasi isi request (422 bila tidak valid).
2. Router memeriksa buku di SQLite lewat `aiosqlite` (404 bila tidak terdaftar).
3. `potong_stok` masuk ke blok `async with stok_lock`, membaca dan mengurangi stok; bila stok kurang dikembalikan 409 Conflict.
4. Transaksi disimpan berstatus `pending` dan respons 202 dikirim seketika.
5. Setelah respons terkirim, `BackgroundTasks` menjalankan `proses_transaksi_background`.
6. Pekerja mengubah status ke `processing`, menunggu `asyncio.sleep(3)`, lalu ke `completed` (atau `failed` dan stok dikembalikan bila galat).
7. Klien memantau perubahan status melalui `GET /transactions/{id}`.

3.3 Daftar endpoint

Tabel 1. Endpoint API Toko Buku

Method	Path	Deskripsi fungsi
GET	`/books?skip=0&limit=10`	Daftar buku dengan paginasi
GET	`/books/{id}`	Detail satu buku (404 bila tidak ada)
POST	`/books`	Tambah buku, validasi Pydantic (201)
PUT	`/books/{id}`	Perbarui buku dan kembalikan data terbaru
DELETE	`/books/{id}`	Hapus permanen (hard delete)
POST	`/transactions`	Buat transaksi: 202 pending; 404 buku tak ada; 409 stok habis
GET	`/transactions?status=`	Daftar transaksi, dapat difilter status
GET	`/transactions/{id}`	Detail transaksi dan status terkini

4. Hasil dan Pembahasan

4.1 Pengujian race condition

```
PS C:\Users\Yasin> cd "E:\Semester 7\Komter\py\tokobuku"
PS E:\Semester 7\Komter\py\tokobuku> Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
PS E:\Semester 7\Komter\py\tokobuku> venv\Scripts\activate
(venv) PS E:\Semester 7\Komter\py\tokobuku> python test_race.py
Respon Request 1: 202 {'id': 1, 'status': 'pending', 'message': 'Transaksi diproses'}
Respon Request 2: 202 {'id': 2, 'status': 'pending', 'message': 'Transaksi diproses'}
Stok akhir: -1
```

Gambar 2. Tahap 1 (tanpa lock): kedua request berhasil (202) dan stok akhir −1 (overselling)

```
(venv) PS E:\Semester 7\Komter\py\tokobuku>    python test_race.py
Respon Request 1: 202 {'id': 3, 'status': 'pending', 'message': 'Transaksi diproses'}
Respon Request 2: 409 {'detail': 'Stok habis / tidak mencukupi'}
Stok akhir: 0
(venv) PS E:\Semester 7\Komter\py\tokobuku>
```

Gambar 3. Tahap 2 (dengan lock): satu request 202 dan satu request 409 Conflict, stok akhir 0

Pada Tahap 1, jeda `asyncio.sleep(0.01)` menyerahkan kendali ke event loop sehingga kedua request sama-sama membaca stok = 1 sebelum salah satunya menulis. Keduanya lolos dan mengurangi stok, sehingga stok akhir −1 (overselling). Pada Tahap 2, blok `async with stok_lock` hanya mengizinkan satu coroutine masuk ke bagian baca–cek–tulis pada satu waktu. Request kedua menunggu, membaca stok 0, lalu ditolak (409 Conflict).

4.2 Hasil benchmark 1 worker vs 4 worker

Tabel 2. Perbandingan performa (median dari 3 kali pengujian)

Skenario	Worker	Total Waktu	Throughput	Rata-rata Latensi
GET /books	1	0,23 s	442,23 req/s	38 ms
GET /books	4	0,17 s	573,83 req/s	27 ms
POST /transactions	1	2,26 s	22,10 req/s	417 ms
POST /transactions	4	1,58 s	31,64 req/s	284 ms

Tabel 3. Hasil tiap pengujian (waktu s / req/s / latensi ms); semua tanpa kegagalan. Satu percobaan awal gagal karena server belum siap dan tidak dihitung.

Skenario	Worker	Run 1	Run 2	Run 3
GET /books	1	0,25 / 394,94 / 40	0,18 / 566,10 / 31	0,23 / 442,23 / 38
GET /books	4	0,17 / 599,38 / 27	0,17 / 573,83 / 26	0,21 / 466,90 / 36
POST /transactions	1	2,83 / 17,64 / 530	2,26 / 22,10 / 417	2,11 / 23,71 / 391
POST /transactions	4	1,58 / 31,64 / 274	9,05 / 5,53 / 1766	1,57 / 31,78 / 284

4.3 Analisis performa

POST /transactions paling terpengaruh oleh penambahan worker. Throughput naik dari 22,10 menjadi 31,64 req/s (sekitar +43%) dan latensi turun dari 417 ms menjadi 284 ms (sekitar −32%). Pada 1 worker, semua request tulis berbaris pada satu `stok_lock` dan satu event loop, sehingga latensi menumpuk. Dengan 4 worker terdapat 4 lock terpisah (satu per proses) sehingga beberapa critical section dapat berjalan paralel. Latensinya hanya ratusan milidetik (bukan 3 detik) karena `asyncio.sleep(3)` berjalan di `BackgroundTasks` setelah respons dikirim.

GET /books hanya membaik sedikit. Throughput naik dari 442,23 menjadi 573,83 req/s (sekitar +30%) dan latensi dari 38 ms menjadi 27 ms. Namun rentang antar-pengujian saling tumpang tindih (395–566 req/s untuk 1 worker, 467–599 req/s untuk 4 worker) dan tiap pengujian hanya berlangsung sekitar 0,2 detik, sehingga selisihnya belum dapat disimpulkan kuat. Endpoint ini ringan dan I/O-bound sehingga satu event loop sudah cukup; proses tambahan menambah overhead dan Locust berbagi CPU dengan server.

Hasil 4 worker kurang stabil. Pada write 4 worker, pengujian ke-2 menghasilkan 9,05 s (5,53 req/s; latensi rata-rata 1766 ms), jauh di luar dua pengujian lainnya. Dugaan penulis: beberapa proses berebut menulis ke satu berkas SQLite yang hanya mengizinkan satu penulis pada satu waktu; dugaan ini belum diuji, sehingga laporan memakai median. Saat start 4 worker di Windows, satu proses anak sempat mencetak `OSError WinError 10022` dan diganti otomatis oleh Uvicorn; 4 proses anak tetap berjalan.

5. Kesimpulan

API Toko Buku memenuhi spesifikasi: CRUD ber-Pydantic, transaksi asinkron (`pending` lalu `completed` lewat `BackgroundTasks`), dan pemotongan stok yang aman. Percobaan membuktikan `asyncio.Lock` mencegah overselling (stok akhir −1 tanpa lock menjadi 0 dengan lock, dan satu request ditolak 409). Penambahan worker dari 1 ke 4 menaikkan throughput tulis sekitar 43% dan menurunkan latensinya sekitar 32%, sedangkan endpoint baca hanya membaik sedikit. Keterbatasan: lock hanya berlaku per proses, SQLite menjadi bottleneck penulisan, dan pengujian dilakukan pada satu mesin dengan request yang sedikit.

Referensi

[1] FastAPI, fastapi.tiangolo.com [2] Python asyncio, docs.python.org/3/library/asyncio.html [3] aiosqlite, aiosqlite.omnilib.dev [4] Uvicorn, www.uvicorn.org [5] Locust, locust.io [6] Modul praktikum dan materi perkuliahan